

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE:CSA51

COURSE NAME: Cryptography and Network Security

COURSE OUTCOME COVERED

CO3: *Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation.CO (BL4 , BL5)*

Assessment Tool Weightage: 20%

QUESTIONS: 5

TOTAL MARKS: 25

Annexure A CASE STUDY

Code of Conduct:

*I Jeevakrishna. R (192511108) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

1. WEAK PASSWORD HASHING IN WEB APPLICATION

CASE DESCRIPTION

A web application stores user passwords using unsalted SHA-1 hashes. After a security breach, attackers quickly crack multiple passwords using precomputed tables. The case requires analysis of how the lack of salting and the use of SHA-1 contributed to the vulnerability and a recommendation for a secure password-hashing strategy.

PROBLEM IDENTIFICATION

The major security problems in the application are:

- The application uses SHA-1 for password storage.
- Passwords are stored without a unique salt.
- The same password produces the same hash for different users.
- Attackers can use precomputed tables to identify commonly used passwords.
- Since SHA-1 is a fast hashing algorithm, attackers can perform password guesses efficiently.
- A database breach can therefore expose many user accounts to password-cracking attacks.

ANALYSIS

A cryptographic hash function converts input data into a fixed-length value called a hash or digest. Instead of storing a user's original password, a secure application should store a password hash and use it during login verification.

In the given case, the application uses the following approach:

Password → SHA-1 → Hash → Database

The main weakness is that the password is hashed directly without a salt. Therefore, the same password always produces the same SHA-1 hash.

For example, if two users choose the password password123, their stored hashes will be identical.

This creates a serious security problem because an attacker who obtains the password database can identify accounts that use the same password.

EFFECT OF LACK OF SALTING

A **salt** is a unique random value added to a password before password hashing. A separate salt should be generated for every password.

Without salting:

Password → SHA-1 → Same Hash

With salting:

Password + Unique Salt → Password Hash

For example:

USER PASSWORD SALT RESULT

User A password123 Salt A Hash A

User B password123 Salt B Hash B

Although both users have the same password, their hashes will be different because different salts are used.

The salt does not need to be kept secret. Its purpose is to make each password-hashing operation unique and to prevent attackers from efficiently using one precomputed hash against multiple accounts.

PRECOMPUTED TABLE ATTACK

A precomputed table contains previously calculated hashes for commonly used passwords. In an unsalted password database, an attacker can compare stolen hashes against these precomputed values.

For example, an attacker may have a collection of values corresponding to common passwords such as:

- password123
- 123456
- qwerty
- welcome123

If the attacker finds a matching hash in the stolen database, the corresponding password can potentially be identified.

The absence of salting makes this attack much more effective because the same password generates the same hash regardless of which user selected it.

Therefore, the lack of salting directly contributed to the rapid cracking of multiple passwords mentioned in the case.

WHY SHA-1 IS INAPPROPRIATE

SHA-1 is a **general-purpose cryptographic hash function**. It is designed to calculate hash values efficiently.

For password storage, however, an algorithm should make password guessing sufficiently expensive for an attacker. A fast hashing process allows an attacker to test a large number of possible passwords quickly.

Therefore, using SHA-1 directly for password storage is inappropriate.

The application should use a dedicated password-hashing algorithm rather than a fast general-purpose hash.

RECOMMENDED SECURE PASSWORD-HASHING STRATEGY

The application should replace unsalted SHA-1 with a dedicated password-hashing algorithm such as **Argon2id**.

A secure password-storage process should be:

1. The user creates a password.
2. The system generates a unique random salt.
3. The password and salt are processed using Argon2id.
4. The resulting password hash is stored.
5. The salt and required hashing parameters are also stored.
6. The original password is never stored.

The stored information should therefore contain the password hash, salt and required parameters, rather than the plaintext password.

During login, the entered password is processed using the stored salt and parameters. The resulting value is then compared with the stored password hash.

WHY ARGON2ID IS SUITABLE

Argon2id is designed specifically for password hashing and can be configured to require significant computational and memory resources.

This is important because password attackers may attempt thousands or millions of guesses. Making each guess more expensive increases the attacker's computational cost.

The use of Argon2id together with a unique salt provides much stronger protection than unsalted SHA-1.

ADVANTAGES OF THE RECOMMENDED APPROACH

- Provides a password-specific hashing mechanism.
- Supports the use of a unique salt for every password.
- Makes large-scale password guessing more expensive.

- Prevents identical passwords from producing identical stored hashes when different salts are used.
- Reduces the effectiveness of precomputed password tables.
- Provides a stronger approach to password storage after a database breach.

ADDITIONAL SECURITY MEASURES

Password hashing should be combined with other security controls.

1. UNIQUE RANDOM SALTS

Every password should have a different randomly generated salt.

2. RATE LIMITING

The application should restrict repeated login attempts. This reduces the effectiveness of automated online password-guessing attacks.

3. MULTI-FACTOR AUTHENTICATION

MFA should be used for sensitive accounts. Even if an attacker obtains a password, an additional authentication factor can provide another layer of protection.

4. SECURE PASSWORD RESET

Password-reset mechanisms should use secure, time-limited reset tokens and should never expose the original password.

5. ACCESS CONTROL

Access to the password database should be restricted to authorized systems and personnel.

6. SECURITY MONITORING

The application should monitor unusual login attempts and other suspicious authentication activity.

COMPARISON OF EXISTING AND RECOMMENDED SYSTEM

FEATURE	EXISTING SYSTEM	RECOMMENDED SYSTEM
Hashing algorithm	Unsalted SHA-1	Argon2id
Salt	Not used	Unique salt for every password
Hashing type	General-purpose	Password-specific
Same password across users	Same hash	Different hashes
Precomputed attack resistance	Low	Much stronger
Password storage	Unsalted hash	Hash + salt + parameters
Additional protection	Limited	Rate limiting + MFA + monitoring
Overall security	Weak	Significantly improved

JUSTIFICATION

The recommended strategy addresses both major causes of the vulnerability.

Replacing SHA-1:

SHA-1 should be replaced with a dedicated password-hashing algorithm such as Argon2id. This makes password-guessing attacks more computationally expensive compared with using a fast general-purpose hash.

Adding unique salts:

A unique random salt should be generated for every password. This ensures that identical passwords produce different stored hashes and significantly reduces the usefulness of precomputed tables.

Protecting against database breaches:

Even if an attacker obtains the password database, a properly configured password-hashing scheme makes recovering the original passwords more difficult.

Adding additional authentication controls:

Rate limiting and MFA provide protection beyond password hashing and help reduce the impact of stolen or guessed passwords.

CONCLUSION

The vulnerability in the web application was caused by the combination of unsalted password storage and the use of SHA-1. The absence of salts allowed attackers to use precomputed tables effectively, while the fast nature of SHA-1 made large-scale password guessing more practical.

The application should replace SHA-1 with a dedicated password-hashing algorithm such as Argon2id and use a unique random salt for every password. The system should store the password hash, salt and required parameters rather than the original password.

Additional measures such as rate limiting, multi-factor authentication, secure password-reset mechanisms, access control and security monitoring should also be implemented.

Therefore, the combination of Argon2id, unique salts and additional security controls provides a significantly stronger password-protection strategy and reduces the risk of successful password-cracking attacks.

2. DIGITAL SIGNATURE COMPROMISE IN FINANCIAL SYSTEM

CASE DESCRIPTION

A financial institution uses digital signatures for transaction approval. An attacker gains access to a user's private key and uses it to perform unauthorized transactions. The case requires evaluating the effect of the private-key compromise on authentication and non-repudiation and proposing suitable mitigation strategies.

PROBLEM IDENTIFICATION

The main problem is the compromise of the user's private key. In a digital signature system, the private key is used to sign transactions, while the corresponding public key is used to verify the signature.

Since the attacker has obtained the private key, they can generate signatures that may successfully pass verification. This can allow unauthorized transactions to appear as if they were approved by the legitimate user.

The major risks are:

- Impersonation of the legitimate user
- Unauthorized financial transactions
- Loss of confidence in the compromised key
- Weakening of authentication
- Difficulty establishing responsibility for disputed transactions

IMPACT ON AUTHENTICATION

Digital signatures help authenticate the source of a transaction by allowing the receiver to verify that the corresponding private key was used.

Under normal conditions, only the legitimate user should have access to the private key. However, after compromise, the attacker also possesses the key.

Therefore, the system cannot reliably distinguish between a transaction signed by the legitimate user and one signed by the attacker.

The attacker may sign unauthorized transactions, impersonate the legitimate account holder and cause financial loss.

IMPACT ON NON-REPUDIATION

Non-repudiation provides evidence that a particular user or entity signed a transaction. It is an important property in financial systems because transactions may need to be verified later.

However, when a private key is compromised, an attacker can use the stolen key to create a valid signature. The legitimate user can then claim that they did not authorize the transaction and that their private key had been stolen.

Therefore, the compromise of the private key weakens the practical assurance of non-repudiation for transactions made after the compromise.

MITIGATION STRATEGIES

Security Measure	Purpose
Secure private-key storage	Prevent unauthorized access to the private key
Hardware Security Module (HSM)	Provide stronger protection for sensitive private keys
Multi-factor authentication	Add another layer of authentication
Certificate revocation	Prevent continued trust in a compromised certificate
Key rotation	Replace compromised keys with new keys
Transaction monitoring	Detect suspicious or unauthorized transactions
Access control	Restrict signing operations to authorized users

PRIVATE-KEY PROTECTION

The private key should be stored securely and should never be exposed to unauthorized users or applications. For a financial institution, a Hardware Security Module (HSM) can be used to protect highly sensitive cryptographic keys.

Access to the signing system should also be restricted using strong authentication and authorization controls.

CERTIFICATE REVOCATION AND KEY ROTATION

Once a private-key compromise is detected, the associated certificate should be revoked immediately so that it is no longer trusted.

A new cryptographic key pair should then be generated and used for future transactions. This prevents the compromised key from continuing to be used by the attacker.

The institution should also investigate transactions created using the compromised key to identify possible unauthorized activity.

ADDITIONAL SECURITY CONTROLS

Multi-factor authentication should be used for high-value or sensitive transactions. Even if the private key is compromised, additional verification can help prevent unauthorized approvals.

The institution should also use transaction monitoring and audit logs to identify unusual activities such as large transfers, repeated transactions or unexpected changes in user behavior.

JUSTIFICATION

No single security mechanism is sufficient to protect a digital-signature system after a private-key compromise. Secure key storage protects the private key, while MFA provides an additional authentication layer.

Certificate revocation prevents continued reliance on the compromised certificate, and key rotation provides a fresh cryptographic identity. Transaction monitoring and audit logs help detect and investigate suspicious activities.

Therefore, combining these mechanisms provides a defense-in-depth approach and significantly reduces the risk of unauthorized financial transactions.

CONCLUSION

The compromise of a user's private key allows an attacker to create apparently valid digital signatures and perform unauthorized transactions. This directly affects authentication because the attacker can impersonate the legitimate user, and it weakens non-repudiation because the legitimate user may dispute transactions created using the stolen key.

The financial institution should protect private keys using secure storage such as HSMs, implement multi-factor authentication, revoke compromised certificates, rotate keys and continuously monitor transactions. These measures provide stronger protection for digital signatures and help maintain the security and trustworthiness of the financial system.

3. KERBEROS AUTHENTICATION FAILURE IN DISTRIBUTED SYSTEM CASE DESCRIPTION

A distributed system using Kerberos experiences frequent authentication failures because of time synchronization issues between servers. The case requires analyzing how time synchronization affects Kerberos authentication and suggesting solutions to maintain reliable and secure authentication.

PROBLEM IDENTIFICATION

The main problem is the difference in system time between the client, authentication server and other servers participating in Kerberos authentication.

Kerberos uses time-sensitive tickets and timestamps as part of its authentication mechanism. Therefore, if the clocks of different systems are not properly synchronized, valid authentication requests may be rejected.

The major issues are:

- Frequent authentication failures
- Ticket rejection
- Service access problems
- Incorrect timestamp validation
- Reduced reliability of the authentication system

ROLE OF TIME IN KERBEROS

Kerberos uses tickets that contain time-related information. These timestamps help the system determine whether an authentication request is still valid.

A ticket is accepted only when its timestamp falls within the permitted time difference between the participating systems.

For example, if the client and server clocks differ significantly, the server may consider the ticket invalid even though the user's credentials are correct.

Therefore, accurate time synchronization is an important requirement for proper Kerberos operation.

HOW TIME SYNCHRONIZATION AFFECTS AUTHENTICATION

Suppose the client has the following time:

Client: 10:00 AM

But the authentication server has:

Server: 10:10 AM

If this difference exceeds the permitted clock-skew value, the authentication request may be rejected.

This results in:

- Users being unable to log in.
- Valid tickets being rejected.
- Applications being unable to access required services.
- Repeated authentication failures across the distributed system.

Thus, time synchronization problems can directly affect the availability and reliability of Kerberos authentication.

SECURITY IMPORTANCE OF TIME SYNCHRONIZATION

Time synchronization is not only required for convenience. It also supports the security of Kerberos. Kerberos uses time limits on authentication information to reduce the possibility of **replay attacks**, where an attacker attempts to reuse previously captured authentication information.

If the system accepts authentication information for an excessively long period, the opportunity for replay attacks may increase.

Therefore, the system must maintain accurate clocks while also maintaining an appropriate clock-skew policy.

SOLUTIONS

SECURITY MEASURE	PURPOSE
NTP synchronization	Keeps system clocks synchronized
Reliable time servers	Provides accurate time to clients and servers
Clock monitoring	Detects time drift and synchronization failures
Multiple time sources	Provides reliability if one source becomes unavailable
Appropriate clock-skew configuration	Allows small clock differences without unnecessarily increasing the security window
Secure time infrastructure	Prevents unauthorized manipulation of system time

NTP-BASED SYNCHRONIZATION

The distributed system should use a reliable **Network Time Protocol (NTP)** infrastructure.

All Kerberos-related systems should synchronize their clocks with trusted time sources. Regular synchronization reduces clock differences between clients and servers.

Administrators should also monitor synchronization status and detect excessive clock drift.

CLOCK-SKEW CONFIGURATION

A suitable clock-skew value should be configured to allow small differences between system clocks. However, the allowed difference should not be made unnecessarily large. Increasing the permitted time difference can increase the period during which captured authentication information may potentially be reused.

Therefore, the system should maintain accurate clocks instead of simply allowing a very large clock difference.

SECURE TIME MANAGEMENT

The time infrastructure itself should be protected. If an attacker can manipulate the system clock, authentication processes may be affected.

The organization should therefore:

- Restrict access to time servers.
- Monitor changes to system time.
- Use trusted time sources.
- Maintain multiple reliable time sources.
- Detect unusual clock changes.

JUSTIFICATION

Time synchronization is essential for reliable Kerberos authentication because the system uses timestamps and time-limited authentication information.

Using NTP, monitoring clock drift and maintaining reliable time sources can prevent unnecessary authentication failures. At the same time, maintaining an appropriate clock-skew value helps preserve the security benefits of time-based authentication.

Therefore, the best approach is to combine accurate time synchronization, secure time management and appropriate clock-skew configuration rather than simply increasing the allowed time difference.

CONCLUSION

The authentication failures in the distributed system are caused by improper time synchronization between Kerberos participants. Since Kerberos depends on time-sensitive authentication information, significant clock differences can cause valid authentication requests and tickets to be rejected.

4. CERTIFICATE AUTHORITY BREACH IN PKI SYSTEM

CASE DESCRIPTION

A Certificate Authority (CA) is compromised, resulting in the issuance of fake X.509 certificates for trusted domains. The case requires evaluating the impact of the breach on system trust and secure communication and proposing mechanisms to detect and mitigate such attacks in a PKI environment.

PROBLEM IDENTIFICATION

The main problem is the compromise of the Certificate Authority.

A CA is a trusted entity in a Public Key Infrastructure (PKI). It issues and digitally signs certificates that establish a relationship between an identity or domain and its public key.

If an attacker compromises a CA, they may be able to issue fraudulent certificates for legitimate domains.

The major risks are:

- Fake certificates being accepted as trusted.
- Impersonation of legitimate websites or services.
- Man-in-the-middle attacks.
- Interception of secure communication.
- Loss of trust in the affected CA.

ROLE OF X.509 CERTIFICATES

An X.509 certificate is used to associate a domain or identity with a public key.

A simplified certificate contains information such as:

- Domain or subject identity
- Public key
- Certificate validity period
- Certificate issuer
- Digital signature of the CA

Normally, the CA's digital signature allows a client to trust that the certificate was issued by a trusted authority.

IMPACT OF CA COMPROMISE

When a CA is compromised, an attacker may create a fake certificate for a trusted domain.

For example, a legitimate domain may have a genuine certificate, but the attacker can obtain another certificate for the same domain from the compromised CA.

If the fraudulent certificate is trusted by the victim's system, the attacker may impersonate the legitimate service.

This can lead to:

- Man-in-the-middle attacks
- User credential theft
- Interception of sensitive information
- Impersonation of trusted websites
- Loss of confidentiality and trust

IMPACT ON SECURE COMMUNICATION

Normally, a client verifies the server's certificate before establishing secure communication.

If the certificate is fraudulent but signed by a compromised trusted CA, the client may accept it as valid.

This creates a serious security problem because the attacker may position themselves between the client and the legitimate service.

Sensitive information such as login credentials, personal information or other confidential data may then be exposed.

DETECTION MECHANISMS

MECHANISM	PURPOSE
Certificate Transparency	Helps identify unexpectedly issued certificates
Certificate monitoring	Detects suspicious certificates issued for an organization's domains
Certificate revocation	Invalidates fraudulent or compromised certificates
CRL	Provides a list of revoked certificates
OCSP	Allows checking whether a certificate has been revoked

MITIGATION STRATEGIES

CERTIFICATE TRANSPARENCY

Certificate Transparency provides publicly auditable records of certificate issuance. Organizations can monitor these records to identify unexpected certificates issued for their domains.

If a suspicious certificate is detected, the organization can investigate and take corrective action.

CERTIFICATE REVOCATION

Fraudulent certificates should be revoked as quickly as possible.

Certificate Revocation Lists (CRLs) and **Online Certificate Status Protocol (OCSP)** can be used to check whether certificates are still valid.

Revocation reduces the period during which a fraudulent certificate can be trusted.

PROTECTING CA PRIVATE KEYS

The CA's private key is extremely sensitive because it is used to sign certificates.

Therefore, CA private keys should be strongly protected using:

- Secure cryptographic hardware
- Strict access controls
- Strong authentication
- Regular security monitoring

Unauthorized access to CA signing infrastructure should be prevented.

CERTIFICATE MONITORING

Organizations should continuously monitor certificates issued for their domains.

Unexpected certificates can indicate:

- CA compromise
- Unauthorized certificate issuance
- Domain impersonation attempts

Early detection allows certificates to be investigated and revoked quickly.

JUSTIFICATION

A PKI system depends on trust in its Certificate Authorities. Therefore, protecting CA infrastructure and detecting fraudulent certificate issuance are essential.

Certificate Transparency improves visibility into certificate issuance, while CRL and OCSP mechanisms help identify certificates that should no longer be trusted.

Strong protection of CA private keys reduces the possibility of attackers issuing fraudulent certificates in the first place.

Using these mechanisms together provides a stronger defense than depending on a single security control.

CONCLUSION

A compromised Certificate Authority can seriously damage the trust model of a PKI system because attackers may obtain fake X.509 certificates for trusted domains. This can enable impersonation, man-in-the-middle attacks and interception of secure communication.

To mitigate the threat, organizations should use Certificate Transparency, certificate monitoring, CRL, OCSP, strong CA private-key protection and strict access controls. These measures help detect fraudulent certificates, revoke compromised certificates and maintain trust in the PKI environment.

5. REPLAY ATTACK IN AUTHENTICATION PROTOCOL

CASE DESCRIPTION

A network authentication protocol is vulnerable to replay attacks because an attacker can capture a valid authentication message and retransmit it later to gain unauthorized access. The case requires analyzing how replay attacks occur and proposing prevention techniques such as nonces and timestamps.

PROBLEM IDENTIFICATION

The main problem is that the authentication protocol does not properly ensure that an authentication message is fresh.

An attacker can capture a valid message exchanged between a user and server and later send the same message again.

The server may accept the repeated message as a new authentication request if it has no mechanism to determine whether the message has already been used.

HOW REPLAY ATTACK OCCURS

A replay attack generally involves three steps:

1. A legitimate user sends a valid authentication message to the server.
2. The attacker captures this message without necessarily understanding its contents.
3. The attacker retransmits the captured message at a later time.

If the protocol does not use freshness mechanisms, the server may accept the repeated message.

EXAMPLE

Suppose a user sends an authentication message containing valid credentials.

The attacker captures the message and stores it.

Later, the attacker sends the same message to the server. If the server only checks whether the message is valid, it may accept it again.

Therefore, simply encrypting or authenticating a message is not always sufficient. The protocol must also ensure that the message is fresh and has not been reused.

EFFECT OF REPLAY ATTACK

A successful replay attack can result in:

- Unauthorized access
- User impersonation
- Repeated execution of requests
- Unauthorized transactions

- Loss of confidentiality and integrity
- Compromise of the authentication system

The severity depends on what actions the captured authentication message allows an attacker to perform.

USE OF NONCES

A nonce is a random or unique value used only once during an authentication session.

The server can generate a nonce and send it to the client. The client includes the nonce in its authenticated response.

For example:

STEP	ACTION
1	Server generates a unique nonce
2	Server sends the nonce to the client
3	Client uses the nonce in its authentication response
4	Server verifies the response and nonce
5	Nonce is marked as used

If an attacker captures the response and tries to replay it later, the old nonce will no longer be valid. Therefore, the replayed message will be rejected.

USE OF TIMESTAMPS

A **timestamp** records the time at which an authentication message was generated.

The server checks whether the timestamp falls within an acceptable time window.

For example, the system may accept only recently generated authentication messages.

If an attacker captures an old message and retransmits it later, the timestamp may fall outside the permitted time window, causing the server to reject the message.

However, timestamps require reasonably accurate clock synchronization between participating systems.

OTHER PREVENTION TECHNIQUES

SEQUENCE NUMBERS

Sequence numbers can be used to identify the order of messages.

The server can reject messages containing an old or previously used sequence number.

CHALLENGE-RESPONSE AUTHENTICATION

The server can send a new challenge for every authentication attempt. The client must generate a valid response to that specific challenge.

This prevents an attacker from successfully reusing an old response.

SESSION IDENTIFIERS

Unique session identifiers can be included in authentication messages to ensure that a message belongs to the correct session.

JUSTIFICATION

The best protection against replay attacks is to ensure that every authentication message contains information proving its freshness.

Nonces are highly effective because a new random value can be generated for every authentication attempt. Timestamps can also prevent old messages from being accepted, provided that the systems maintain accurate time.

For stronger protection, the protocol can combine nonces, timestamps, secure message authentication and session identifiers.

CONCLUSION

A replay attack occurs when an attacker captures a valid authentication message and retransmits it later. If the authentication protocol does not verify the freshness of messages, the server may incorrectly accept the replayed message as a legitimate request.

Using nonces, timestamps, sequence numbers and challenge-response authentication can effectively prevent replay attacks. Among these, a unique nonce for each authentication session provides a strong method for ensuring that previously captured authentication messages cannot be reused.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand